UNIT-3(DEADLOCK)

IntroductionofDeadlockinOperatingSystem LECTURE NO:-1

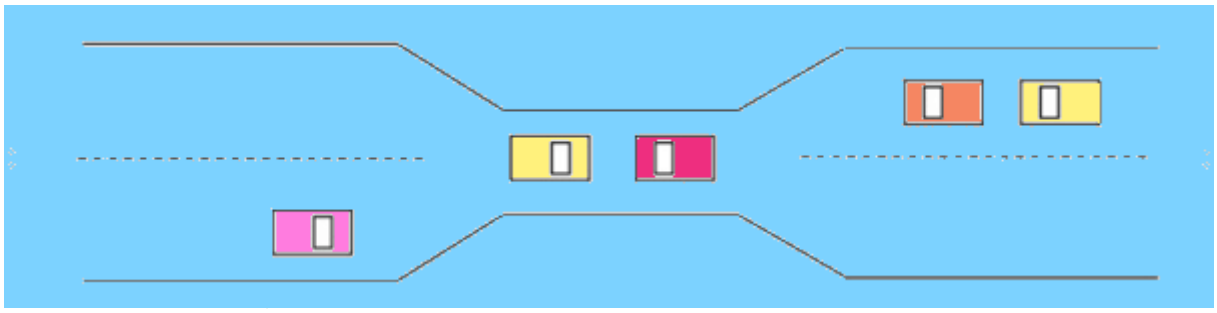
IntroductionofDeadlock

WhatisDeadlock?

Deadlock is a situation that occurs in OS when any process enters a waiting state because another waiting process is holding the demanded resource. Deadlock is a common problem in multi-processing where several processes share a specific type of mutually exclusive resource known as a soft lock or software.

ExampleofDeadlock

- Areal-worldexamplewouldbetraffic,whichisgoingonlyinone direction.
- Here, abridgeisconsideredaresource.
- So,whenDeadlockhappens, it canbeeasilyresolved ifonecarbacksup(Preempt resourcesand rollback).
- Severalcarsmayhavetobebacked upifadeadlock situationoccurs.
- Sostarvationispossible.



Exampleofdeadlock

WhatisCircularwait?

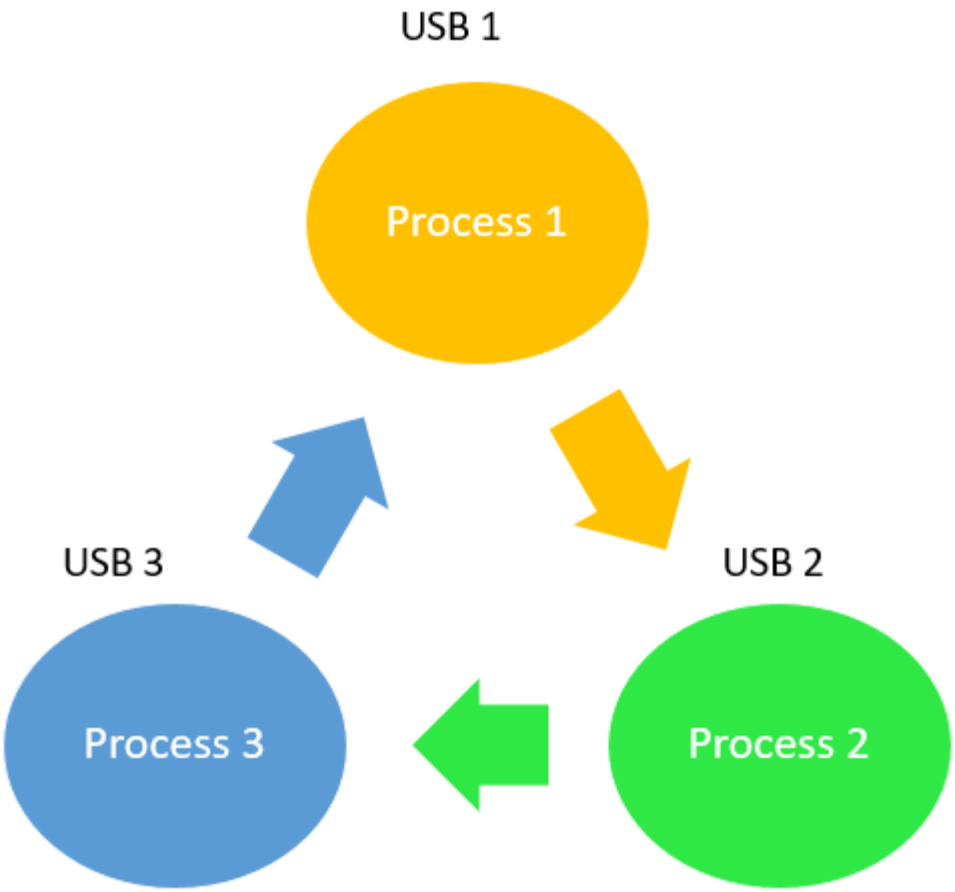
One process is waiting for the resource, which is held by the second process, which is alsowaiting for the resource held by the third process etc. This will continue until the last process is waiting for a resource held by the first process. This creates a circular chain.

`

For example, ProcessAisallocatedResourceBas it isrequestingResourceA. Inthesameway, Process B is allocated Resource A, and it is requesting Resource B. This creates a circular wait loop.

ExampleofCircularwait

For example, a computer has three USB drives and three processes. Each of the three processes able to holds oneofthe USB drives. So, wheneachprocess requests another drive, thethree processes will have the deadlock situationas eachprocess will be waiting for the USB drive to release, which is currently in use. This will result in a circular chain.



Circularwaitexample

DifferenceBetweenStarvationandDeadlock

Here,aresome importantdifferencesbetweenDeadlockandstarvation-

`

Deadlock	Starvation
Thedeadlocksituationoccurswhenoneofthe processes got blocked.	Starvationisasituationwhereallthe lowpriority processes got blocked, and the high priority processes execute.
Deadlockisaninfiniteprocess.	Starvationisalongwaitingbutnotaninfinite process.
EveryDeadlockalwayshas starvation.	Everystarvationdoesn'tnecessarilyhavea deadlock.
Deadlock happens then Mutualexclusion, hold andwait.Here,preemptionandcircularwaitdo not occur simultaneously.	Ithappensduetouncontrolledpriorityand resource management.

AdvantagesofDeadlock

Here,arepros/benefitsofusingDeadlockmethod

- Thissituationworkswellforprocesseswhichperformasingle burstofactivity
- Nopreemptionneededfor Deadlock.
- Convenientmethodwhenappliedtoresourceswhosestatecanbesavedandrestoredeasily
- Feasibletoenforceviacompile-timechecks
- Needsnorun-timecomputationsincetheproblem issolvedinsystemdesign

DisadvantagesofDeadlockmethod

Here,arecons/drawbackofusing deadlockmethod

- Delaysprocess initiation
- Processesmustknowfutureresourceneed
- Pre-emptsmoreoftenthannecessary
- Dis-allows incrementalresource requests
- Inherentpreemptionlosses.

`

Summary-

- DeadlockDefinition: It isasituationthatoccursinOSwhenanyprocessentersawaitingstate because another waiting process is holding the demanded resource
- Circular waitinghappenswhenoneprocessiswaiting fortheresource,whichisheld bythe second process, which is also waiting for the resource held bythe third process etc.
- Adeadlockoccurrencecanbe detectedbytheresource scheduler.
- It'simportanttopreventadeadlockbeforeitcanoccur.
- Aresourcecanbereleasedonlyvoluntarilybythe processholding it afterthatprocesshas finished its task.
- MutualExclusion isa fullformofMutex. It isaspecialtypeofbinarysemaphore whichused for controlling access to the shared resource.
- Holdandwait isaconditionwhereprocessesmust bestoppedfromholdingsingleormultiple resources while simultaneously waiting for one or more others.
- Deadlockavoidance isthesimplest and most usefulmodelthat eachprocessdeclaresthe maximum number of resources of each type that it may need.
- Thedeadlock-avoidancealgorithmhelpsyouto dynamicallyassesstheresource-allocationstate so that there can never be a circular-wait situation.
- Deadlockisaninfiniteprocess,whereasstarvationisa longwaitingbutnotaninfiniteprocess.

MIT

`

UNIT-3(DEADLOCK)
ConditionsforDeadlocks LECTURE NO:-2

ConditionsforDeadlocks

Everyprocess needs some resources to complete its execution. However, the resource is granted in a sequential order.

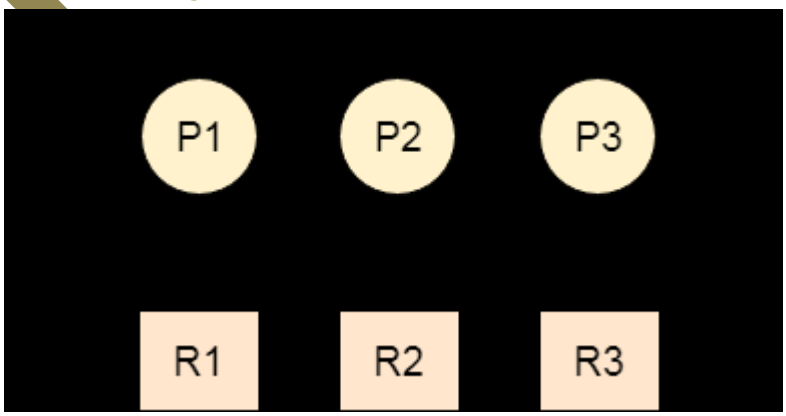
1. Theprocessrequestsforsomeresource.
2. OSgranttheresourceifitisavailableotherwiseletttheprocesswaits.
3. Theprocessuses itandreleaseonthe completion.

ADeadlock is a situationwhere eachofthe computerprocess waits for a resourcewhich is being assigned to some another process. In this situation, none of the process gets executed since the resource it needs, is held by some other process which is also waiting for some other resource to be released.

Let us assume that there are three processes P1, P2 and P3. There are three different resources R1, R2 and R3. R1 is assigned to P1, R2 is assigned to P2 and R3 is assigned to P3.

After some time, P1 demands for R1 which is being used by P2. P1 halts its execution since it can't complete without R2. P2 also demands for R3 which is being used by P3. P2 also stops its execution because it can't continue without R3. P3 also demands for R1 which is being used by P1 therefore P3 also stops its execution.

In this scenario, a cycle is being formed among the three processes. None of the process is progressing and they are all waiting. The computer becomes unresponsive since allthe processes got blocked.



NecessaryconditionsforDeadlocks

1.MutualExclusion

A resource can only be sharedin mutually exclusivemanner.It implies,if twoprocess cannot use the same resource at the same time.

2. HoldandWait

Aprocesswaitsfor someresourceswhileholdinganother resourceatthesame time.

3. Nopreemption

The process which once scheduled will be executed tillthe completion. No other process can be scheduled by the scheduler meanwhile.

4. CircularWait

Alltheprocessesmustbewaitingfortheresourcesinacyclicmannersothatthelast process is waiting for the resource which is being held by the first process.

UNIT-3(DEADLOCK)
ConditionsforDeadlocks LECTURE NO:-3

StrategiesforhandlingDeadlock

1. DeadlockIgnorance

Deadlock Ignorance is the most widely used approach among all the mechanism. This is being used bymanyoperating systems mainlyfor end user uses. Inthis approach, the Operating system assumes that deadlock never occurs. It simply ignores deadlock. This approach is best suitablefor a single end user system where User uses the system only for browsing and all other normal stuff.

There is always a tradeoff between Correctness and performance. The operating systems like Windows and Linux mainly focus upon performance. However, the performance of the system decreases if it uses deadlock handling mechanism all the time if deadlock happens 1 out of 100 times then it is completely unnecessaryto use the deadlock handling mechanism all the time.

In these types of systems, the user has to simply restart the computer in the case of deadlock. Windows and Linux are mainly using this approach.

2. Deadlockprevention

Deadlock happens only when MutualExclusion, hold and wait, No preemption and circular wait holds simultaneously. If it is possible to violate one of the four conditions at any time then the deadlock can never occur in the system.

The idea behind the approach is very simple that we have to fail one of the four conditions but there can be a big argument on its physical implementation in the system.

Wewilldiscussitlaterindetail.

3. Deadlockavoidance

In deadlock avoidance, the operating system checks whether the system is in safe state or in unsafe state at every step which the operating system performs. The process continues until the system is in safe state. Once the system moves to unsafe state, the OS has to backtrack one step.

In simple words, The OS reviewseach allocation so that the allocation doesn'tcause the deadlock in the system.

WewilldiscussDeadlockavoidancelater indetail.

4. Deadlockdetectionandrecovery

This approach let the processes fall in deadlock and then periodically check whether deadlock occur in the systemor not. Ifit occursthen it applies some ofthe recovery methods tothe system to get rid of deadlock.



UNIT-3(DEADLOCK)

DeadlockPrevention
LECTURE NO:-4

DeadlockPrevention

If we simulate deadlock with a table which is standing on its four legs then we can also simulate four legs with the four conditions which when occurs simultaneously, cause the deadlock.

However, if we break one of the legs of the table then the table will fall definitely. The same happens with deadlock, if we can be able to violate one of the four necessaryconditions and don't let them occur together then we can prevent the deadlock.

Let'ssee how we canpreventeachoftheconditions.

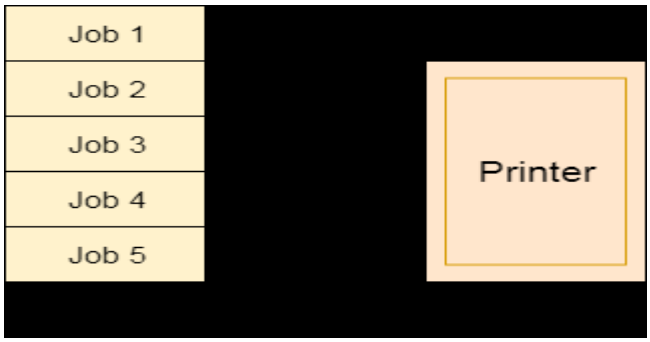
1. MutualExclusion

Mutualsectionfromtheresourcepoint ofview is the fact that aresourcecannever beused bymorethan one process simultaneously which is fair enough but that is the main reason behind the deadlock. If a resource could have been used by more than one process at the same time then the process would have never been waiting for any resource.

However, if we can be able to violate resources behaving in the mutually exclusive manner then the deadlock can be prevented.

Spooling

For a device like printer, spooling can work. There is a memoryassociated with the printer which stores jobs from each of the process into it. Later, Printer collects all the jobs and print each one of them according to FCFS. By using this mechanism, the process doesn't have to wait for the printer and it can continue whatever it was doing. Later, it collects the output when it is produced.



`

Although, Spooling can be an effective approach to violate mutual exclusion but it suffers from two kinds of problems.

1. Thiscannotbeappliedtoeveryresource.
2. After some point of time, there may arise a race condition between the processes to get space in that spool.

We cannot force a resourceto be used bymore thanone process at the same time since it will not befair enough and some serious problems may arise in the performance. Therefore, we cannot violate mutual exclusion for a process practically.

2. HoldandWait

Hold and wait condition lies when a process holds a resource and waiting for some other resource to complete its task. Deadlock occurs because there can be more than one process which are holding one resource and waiting for other in the cyclic order.

However, we have to find out some mechanism by which a process either doesn't hold any resource or doesn't wait. That means, a process must be assigned all the necessary resources before the execution starts. A process must not wait for anyresource once the execution has been started**.**

!(Holdandwait)=!holdor!wait(negationof holdandwaitis,eitheryoudon'thold or you don't wait)

This can be implemented practically if a process declares all the resources initially. However, thissounds very practical but can't be done in the computer system because a process can't determine necessary resources initially.

Process is the set of instructions which are executed by the CPU. Each of the instruction may demand multiple resources at the multiple times. The need cannot be fixed by the OS.

Theproblemwiththeapproachis:

1. Practicallynotpossible.
2. Possibilityofgettingstarvedwillbeincreasesduetothefactthatsomeprocessmayholda resource for a very long time.

3. NoPreemption

Deadlock arises due to the fact that a process can't be stopped once it starts. However, if we take the resource away from the process which is causing deadlock then we can prevent deadlock.

`

This is not a good approach at all since if we take a resource away which is being used by the process then all the work which it has done till now can become inconsistent.

Consider a printer is being used by any process. If wetakethe printer away fromthat process and assign it to some other processthenallthe data which has beenprinted canbecome inconsistent and ineffective and also the fact that the process can't start printing again from where it has left which causes performance inefficiency.

4.CircularWait

To violate circular wait, we can assign a priority number to each ofthe resource. Aprocess can't request for a lesser priorityresource. This ensures that not a single process can request aresourcewhich is being utilized by some other process and

nocyclewillbeformed.

Condition	Approach	Is Practically Possible?
Mutual Exclusion	Spooling	✗
Hold and Wait	Request for all the resources initially	✗
No Preemption	Snatch all the resources	✗
Circular Wait	Assign priority to each resources and order resources numerically	✓

Deadlockavoidance

In deadlock avoidance, the request for any resource will be granted if the resulting state of the system doesn't cause deadlock in the system. The state of the system will continuously be checked for safe and unsafe states.

In order to avoid deadlocks, the process must tell OS, the maximum number of resources a process can request to complete its execution.

`

The simplest and most useful approach states that the process should declare the maximum number of resources of each type it may ever need. TheDeadlock avoidance algorithm examines the resource allocations so that there can never be a circular wait condition.

SafeandUnsafeStates

Theresourceallocationstateofasystemcanbedefinedbytheinstancesofavailableandallocated resources, and the maximum instance of the resources demanded by the processes.

Astateofa systemrecordedatsome randomtimeisshownbelow.

ResourcesAssigned

Process	Type1	Type2	Type3	Type4
A	3	0	2	2
B	0	0	1	1
C	1	1	1	0
D	2	1	4	0

Resourcesstillneeded

Process	Type1	Type2	Type3	Type4
A	1	1	0	0
B	0	1	1	2
C	1	2	1	0
D	2	1	1	2

1. E=(7684)

`

2. P=(6283)
3. A=(1401)

Above tables and vector E, P and A describes the resource allocation state of a system. There are 4 processes and 4 types of the resources in a system. Table 1 shows theinstances of each resourceassigned to each process.

Table 2 shows the instances of the resources, each process still needs. Vector E is the representation of total instances of each resource in the system.

Vector P representsthe instances ofresources that have beenassigned to processes. Vector Arepresents the number of resources that are not in use.

A state of the system is called safe if the system can allocate all the resources requested by all the processes without entering into deadlock.

Ifthe systemcannotfulfilltherequestofallprocessesthenthestateofthesystemiscalledunsafe.

The key of Deadlock avoidance approach is when the requestis made for resources then the request must only be approved in the case if the resulting state is also a safe state.



`



`

UNIT-3(DEADLOCK)

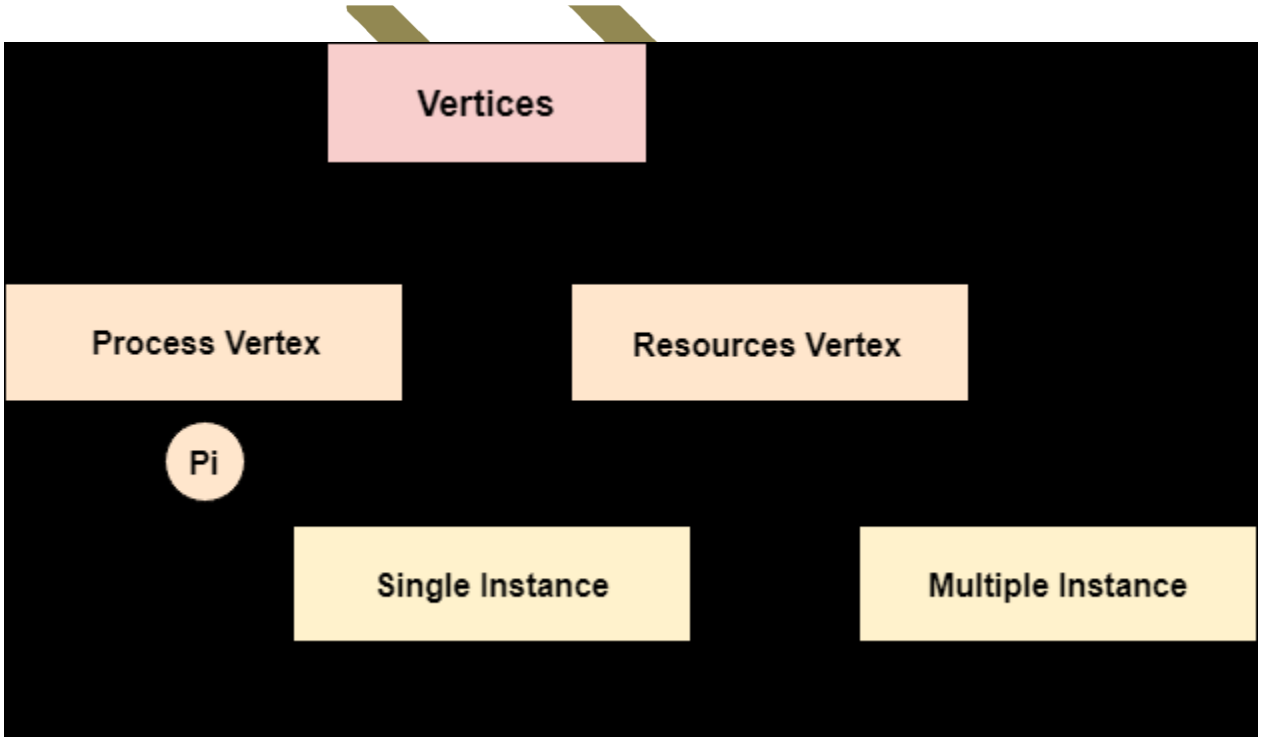
ResourceAllocationGraph LECTURE NO:-5

ResourceAllocationGraph

The resource allocation graph is the pictorial representation of the state of a system. As its namesuggests, the resource allocation graph is the complete information about all the processes which are holding some resources or waiting for some resources.

It also contains the information about allthe instances of all the resources whether theyare available or being used by the processes.

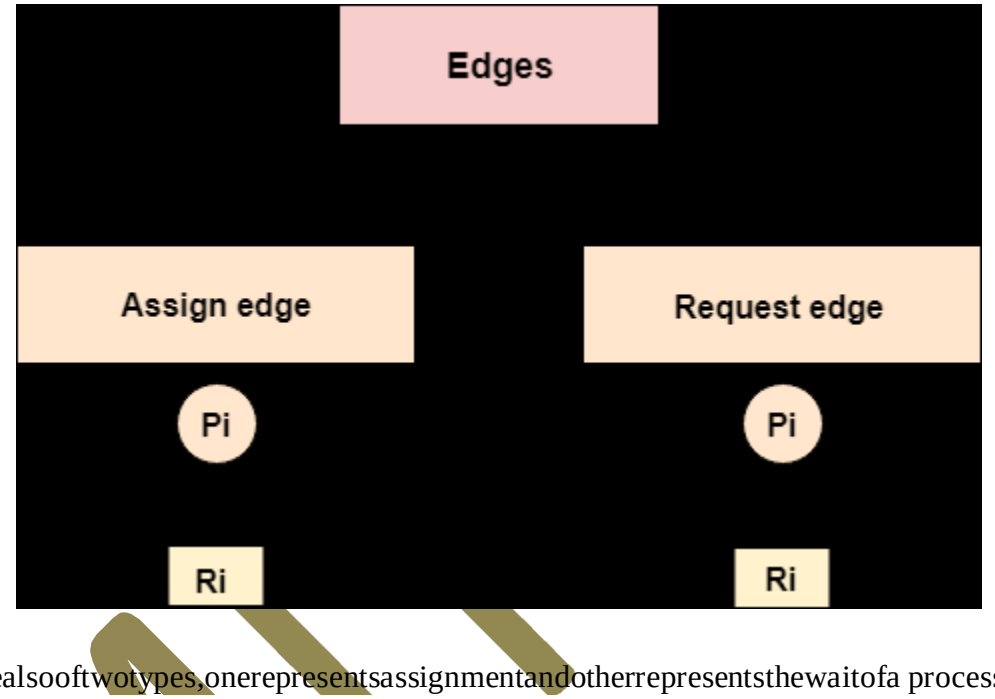
InResource allocationgraph, theprocess is representedbyaCirclewhiletheResourceis representedby a rectangle. Let's see the types of vertices and edges in detail.



`

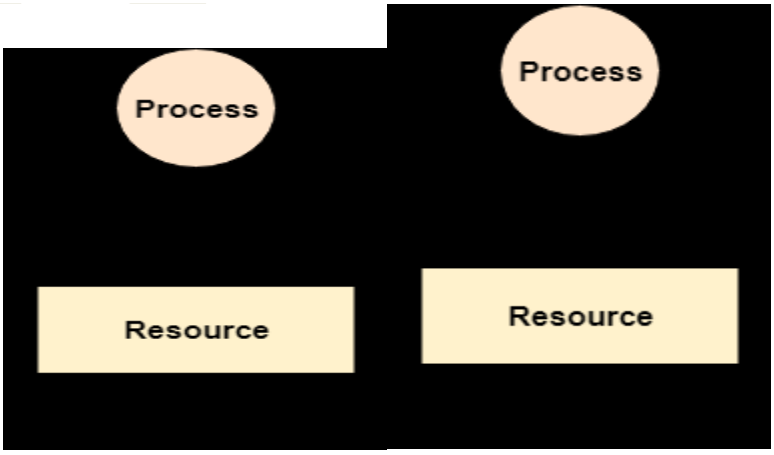
Vertices are mainlyoftwo types, Resource and process. Each ofthemwill be represented bya different shape. Circle represents process while rectangle represents resource.

Aresourcecanhavemorethanoneinstance.Eachinstancewillberepresentedbyadotinsidethe rectangle.



EdgesinRAGarealsooftwotypes,onerepresentsassignmentandotherrepresentsthewaitofa process for a resource. The above image shows each of them.

Aresource isshownasassignedtoa processifthetailofthearrow isattachedtoan



`

instanceto theresourceandtheheadisattachedtoaprocess.

Aprocessisshownaswaiting for aresourceifthe tailofanarrowisattached to theprocess whilethe head is pointing towards the resource.

Example

Let'sconsider3processesP1,P2andP3,andtwotypesofresourcesR1andR2.Theresourcesare having 1 instance each.

According to the graph, R1 is being used byP1, P2 is holding R2 and waiting for R1, P3 is waiting for R1 as well as R2.

Thegraphisdeadlockfreesincenocycleisbeingformedinthegraph.



`

MITC

`

UNIT-3(DEADLOCK)

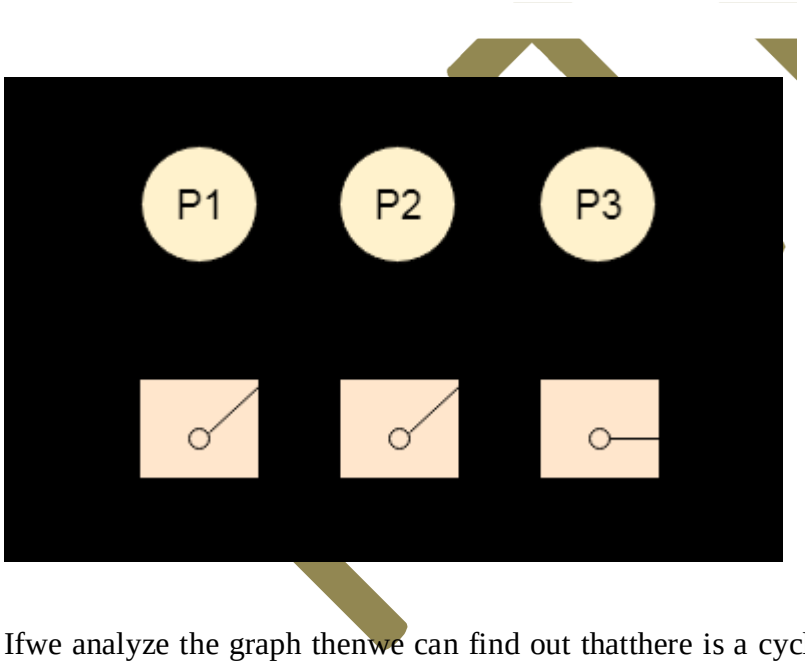
DeadlockDetectionusingRAG
LECTURE NO:-6

DeadlockDetectionusingRAG

Ifacycle is being formed inaResourceallocationgraphwhere alltheresourceshavethesingle instance then the system is deadlocked.

InCaseofResourceallocationgraphwith multi-instancedresourcetypes, Cycle is anecessarycondition of deadlock but not the sufficient condition.

The following example contains three processes P1, P2, P3 and three resources R2, R2, R3. All the resources are having single instances each.



Ifwe analyze the graph thenwe can find out thatthere is a cycle formed in the graph since the system is satisfying all the four conditions of deadlock.

Allocation Matrix

Allocation matrix can be formed by using the Resource allocation graph of a system. In Allocation matrix,an entry will bemadefor each of the resource assigned.For Example,in the followingmatrix, en entry is being made in front ofP1 and below R3 since R3 is assigned to P1.

`

Process	R1	R2	R3
P1	0	0	1
P2	1	0	0
P3	0	1	0

RequestMatrix

Inrequest matrix, anentrywillbe made for eachoftheresourcerequested. As inthe followingexample, P1 needs R1 therefore an entry is being made in front of P1 and below R1.

Process	R1	R2	R3
P1	1	0	0
P2	0	1	0
P3	0	0	1

Avial=(0,0,0)

Neither we are having any resource available in the system nor a process going to release. Each of the process needs at leastsingle resource tocomplete therefore they will continuously be holding each one of them.

Wecannot fulfillthedemandofat least oneprocess using theavailableresourcesthereforethesystemis deadlocked as determined earlier when we detected a cycle in the graph.

`

UNIT-3(DEADLOCK)

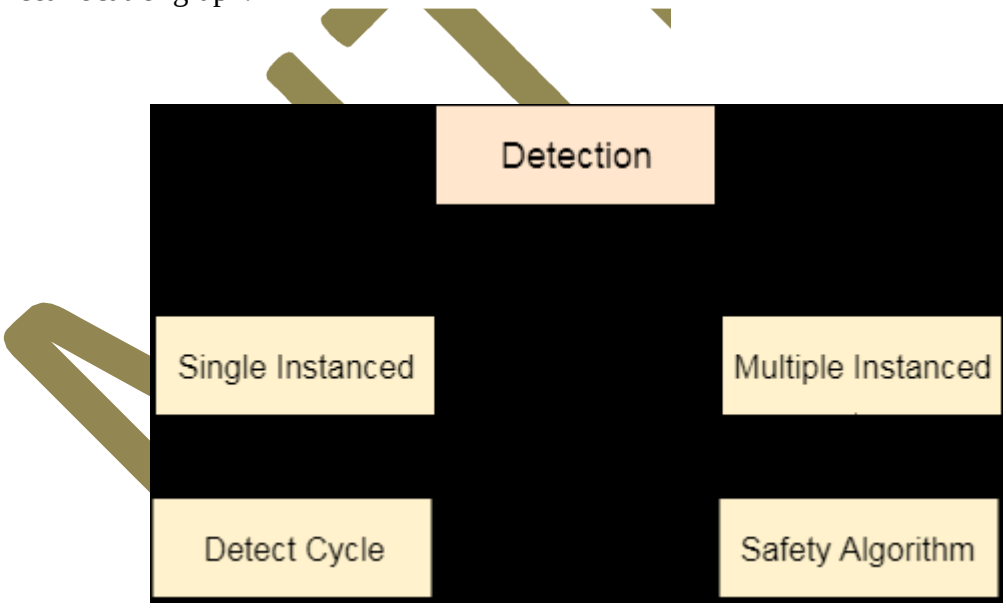
DeadlockDetectionandRecovery

LECTURENO:-7

DeadlockDetectionandRecovery

In this approach, The OS doesn't applyany mechanism to avoid or prevent the deadlocks. Therefore the systemconsiders that the deadlock will definitelyoccur. Inorderto get rid ofdeadlocks, The OS periodicallychecksthesystemforanydeadlock.Incase,itfindsanyofthedeadlockthentheOS willrecoverthe systemusingsomerecoverytechniques.

ThemaintaskoftheOSisdetecting thedeadlocks.The OScandetect thedeadlockswiththehelp of Resourceallocationgraph.



In single instanced resource types, if a cycle is being formed in the systemthen there will definitely be a deadlock. Onthe other hand, in multiple instanced resource type graph, detecting a cycle is not justenough.Wehavetoapplythesafetyalgorithmonthesystembyconvertingtheresource allocationgraphintotheallocationmatrixandrequestmatrix.

Inordertorecoverthe systemfromdeadlocks,eitherOSconsidersresourcesorprocesses.

Preempttheresource

We can snatch one ofthe resources fromthe owner ofthe resource (process) and give it to the other process withthe expectationthat it willcompletetheexecutionandwillreleasethisresourcesooner. Well, choosing a resource which will be snatched is going to be a bit difficult.

Rollbacktoa safestate

System passes through various states to get into the deadlock state. The operating systemcanrollback the system to the previous safe state. For this purpose, OS needs to implement check pointing at every state.

The moment, we get into deadlock, we will rollback all the allocations to get into the previous safe state.

For Process

Killaprocess

Killing a process can solve our problem but the bigger concern is to decide which process to kill. Generally, Operating system kills a process which has done least amount of work until now.

Killallprocess

This is not a suggestible approach but can be implemented if the problem becomes very serious. Killing allprocess will lead to inefficiency in the systembecause allthe processes will execute again from starting.

`



MITC

`

UNIT-3(DEADLOCK)
DeviceManagementinOperatingSystem

LECTURE NO:-8

DeviceManagementinOperatingSystem

Device management in operating system implies the management ofthe I/O devices such as a keyboard, magnetic tape, disk, printer, microphone, USB ports, scanner, camcorder etc.as well as the supporting units like control channels. The basics of I/O devices can fall into 3 categories:

Blockdevice: it storesinformation in fixed-size block, eachone with itsownaddress.Fr example, disks.
Character device: it's delivers or accepts a stream of characters. The individual characters are not addressable. For example printers, keyboards etc.
Networkdevice: Fortransmitting datapackets.

Themainfunctionsofdevicemanagementintheoperatingsystem

An operating system or the OS manages communication with the devices through their respective drivers. The operating system component provides a uniform interface to access devices of varied physical attributes. For device management in operating system:

- Keep tracks of all devices and the program which is responsible to perform this is called I/O controller.
- Monitoring the status of each device such as storage drivers, printers and other peripheraldevices.
- Enforcing preset policies and taking a decision which process gets the device when and for how long.
- Allocates and Deallocates the device in an efficient way.De-allocating them at two levels: at the process level when I/O command has been executed and the device is temporarily released, andat the job level, when the job is finished and the device is permanently released.
- Optimizestheperformanceofindividualdevices**.**

Typesofdevices

`

The OS peripheral devices can be categorized into 3: Dedicated, Shared, and Virtual. The differences among them are the functions of the characteristics of the devices as well as how they aremanaged by the Device Manager.

Dedicateddevices:-

Such type of devices in the **device management in operating system** are dedicated or assigned to only one job at a time until that job releases them. Devices like printers, tape drivers, plotters etc. demandsuch allocation scheme since it would be awkward if severalusers share them at the same point oftime. The disadvantages of such kind f devices s the inefficiency resultingfrom the allocation of the device toa single user for the entire durationofjobexecutioneventhoughthe device is not put to use 100%ofthe time.

Shareddevices:-

These devices can be allocated o several processes. Disk-DASD can be shared among severalprocesses at the same time by interleaving their requests. The interleaving is carefully controlled by the Device Manager and all issues must be resolved on the basis of predetermined policies.

VirtualDevices:-

These devices are the combination of the first two types and they are dedicated devices which are transformed into shared devices. For example, a printer converted into a shareable device via spooling program which re-routes all the print requests to a disk. A print job is not sent straight to the printer, instead, it goes to the disk(spool)until it is fully prepared with all the necessary sequences andformatting, then it goes to the printers. This technique can transform one printer into several virtual printers which leads to better performance and use.

Waystoaccessadeviceindevicemanagementinoperatingsystem

Polling:-

In this, a CPU continuously checks the device status for exchanging data. The plus point is that it is simple and the negative point is– Busy-waiting.

Interrupt-drivenI/Os:-

A device controller notifies the corresponding device driver about the availability of the device. The advantages can be a more efficient use of CPU cycles and the drawbacks can be data copying and movements and slow for character devices-one interrupt per keyboard input.

`

Note:-

Adevice driver is anoperatingsystemcomponent whichis responsible for thehiding fcomplexityofan I/O device so that the OS can access various devices in a uniform manner.

A device controller performs conversion between serial bit stream and a block of bytes, do error correction if necessaryand has 2 maincomponents;a device registersto communicate withthe CPU and a Data buffer that an OS can read or write.

Directmemoryaccess(DMA):-

To perform data movements additional controller bought into use. The benefit of such a method is that CPU is not involved in copying data but a con is that a process cannot access in-transit data.

Doublebuffering:

This methodology access advice makes use of two buffers so that while one is being used, the other is being filled. Such a way is quite popular in graphics and animation so that the viewer does not see the line-by-line scanning.

Twohard-disksconfigurationsindevicemanagementinoperatingsystem

Mobile-headmagneticdiskstorage:-

Both the reading/writing head moves and the disk spins. It is usually formatted on both sides and data is recorded on tracks. Typically there is more than one disk(platter), and they are arranged in a stack on a common spindle and have Read/Write heads n both sides. All of the heads move in unison.

Flexible-headmagneticdiskstorage:-

Its veryfast when compared to its mobile counterpart but it is expensive to the manufacture and has less storage space as the tracks have to be positioned farther apart. They are often used in aircraft or space applications where speed is an important factor.

Opticaldiskstorageindevicemanagementinoperatingsystem

Data can be stored optically than magnetically due to advances in laser technology, for example, CDsand DVDs. Optical disks are usually single continuous spiral track and not tracks as a series ofconcentric circles. Magnetic disks spin at a constant speed so the tracks on other outer rings of magnetic disks have to have larger sectors andthosetracks nearer the center have smalltracks. Onthe otherhand, opticaldisks can change the speed to reading, so the sector size is the same in the outside or inside.

`

A high-intensity laser bean burns indentations in the disk to write to anopticaldisk. These burned areas are called pits(represents 0's) and the unburned flat areas are called lands (represents 1's).

The 3 important performance measures inopticaldisk storagearedatatransfer rate(measures the amount of data that can be read from the disk, unit is megabytes per second-MB/s), average access time (measures how long it takes to move the disk head to a specific place on the disk, unit is milliseconds- ms), and cache size(it saves the datato be re-read again by the user and is measured n kilobytes-KB).

ComponentsofanI/Osystem

The CPU makes a lot ofI/O requests and the I/Ochannels workto coordinate the devices to synchronize the fast speed ofthe CPU withthe slower speeds ofthe I/O devices. The I/O control units are attached to several similar devices and control these devices based on instructions from the channel.



MITC

`

UNIT-3(DEADLOCK)

Disk Scheduling

LECTURENO:-9

DiskScheduling

FCFSSchedulingAlgorithm

It is thesimplest DiskSchedulingalgorithm. It servicestheIOrequestsintheorder inwhichtheyarrive. There is no starvation in this algorithm, every request is serviced.

Disadvantages

- Theschemedoesnotoptimizetheseektime.
- The request may come fromdifferent processes therefore there is the possibilityof inappropriate movement of the head.

Example

Considerthefollowingdiskrequestsequenceforadiskwith100tracks45,21,67,90,4,50,89,52,61, 87,25

Headpointerstartingat50andmovinginleftdirection.Findthenumberofheadmovementsincylinders using FCFS scheduling.

Solution

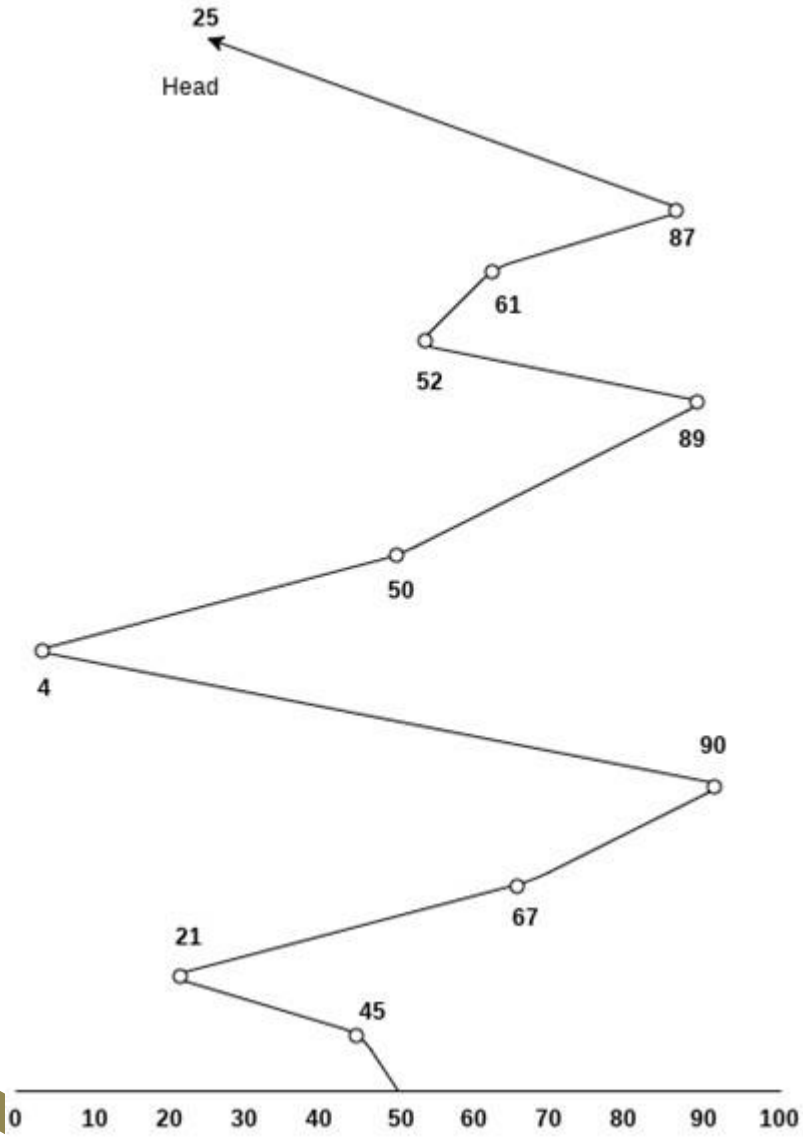
Number ofcylindersmoved bythehead

=(50-45)+(45-21)+(67-21)+(90-67)+(90-4)+(50-4)+(89-50)+(61-52)+(87-61)+(87-25)

=5+24+46+23+86+46+49+9+26+62

=376

`


25
Head
87
61
52
89
50
4
90
67
21
45
0
10
20
30
40
50
60
70
80
90
100



`

UNIT-3(DEADLOCK)

Look Scheduling

LECTURENO:-10

LookScheduling

It is like SCAN scheduling Algorithm to some extant except the difference that, in this scheduling algorithm, the arm of the disk stops moving inwards (or outwards) when no more request in thatdirection exists. This algorithm tries to overcome the overhead of SCAN algorithm which forces diskarm to move in one direction till the end regardless of knowing if any request exists in the direction or not.

Example

Considerthefollowingdiskrequest sequenceforadiskwith100tracks 98,

137, 122, 183, 14, 133, 65, 78

Head pointer starting at 54 and moving in left direction. Find the number of head movements in cylinders using LOOK scheduling.

Numberofcylinderscrossed= 40+51+13++20+ 24+11+4+ 46=209

CLookScheduling

C Look Algorithm is similar to C-SCAN algorithmto some extent. Inthis algorithm, the armofthe disk moves outwards servicing requests until it reaches the highest request cylinder, then it jumps to the lowest request cylinder without servicing any request then it again start moving outwards servicing the remaining requests.

It is different fromC SCAN algorithm inthe sense that, C SCAN force the disk armto move till the last cylinder regardless of knowing whether any request is to be serviced on that cylinder or not.

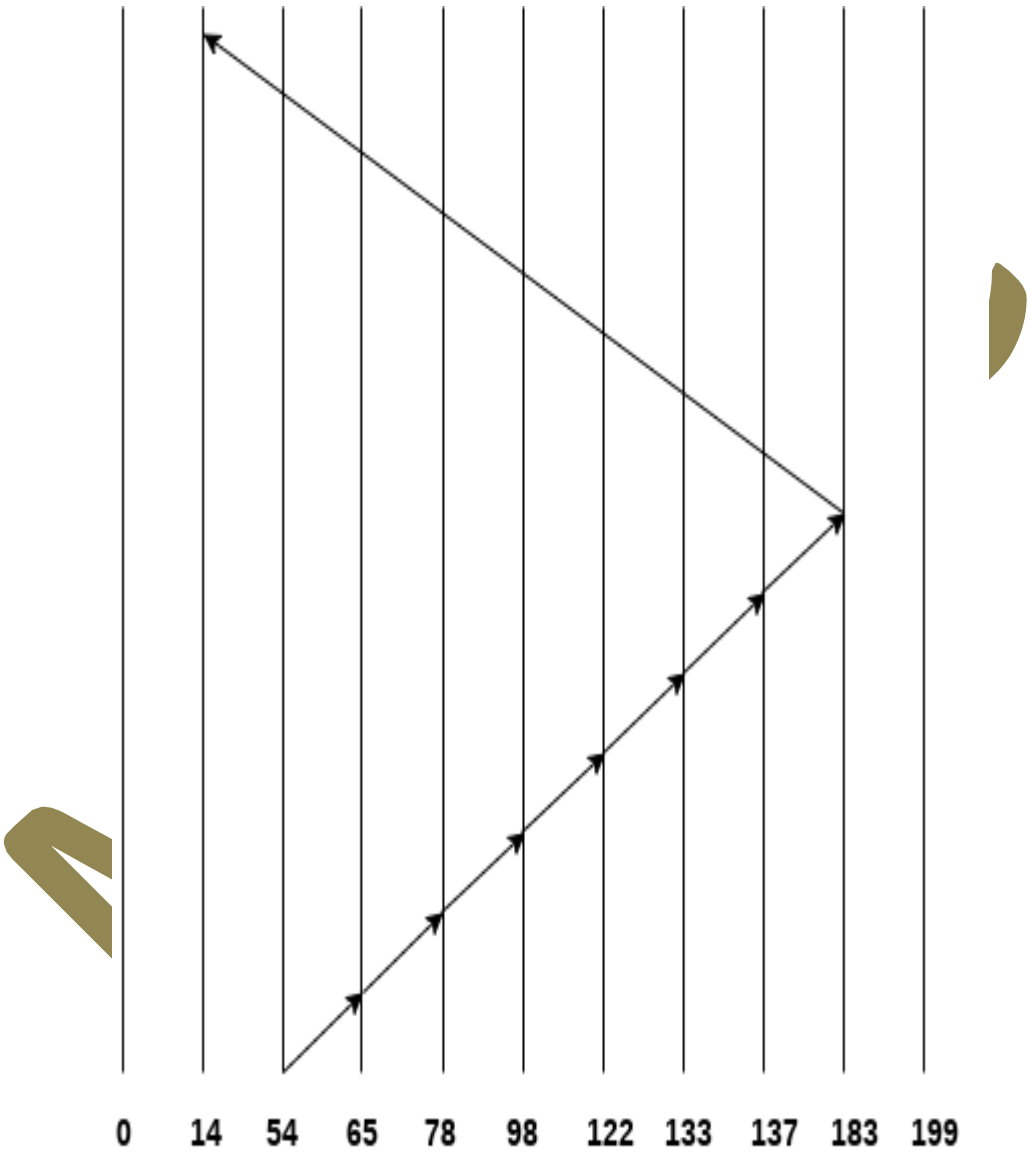
Example

Considerthefollowingdiskrequestsequencefor adiskwith100tracks

98,137,122,183,14,133,65,78

`

Headpointerstartingat54andmovinginleftdirection.Findthenumberofheadmovementsin cylinders using C LOOK scheduling.



Numberofcylinderscrossed=11+13+20+24+11+4+46+169= 298

`

NumericalonSSTFand SCAN

Question:

Suppose the following disk request sequence (track numbers) for adisk with 100 tracks is given: 45, 20, 90, 10, 50, 60, 80 and 70. Assume that the initial positionofthe R/Whead is ontrack50. The additional distance that will be traversed by the R/W head when the Shortest Seek Time First (SSTF) algorithm is used compared to the SCAN (Elevator) algorithm (assuming that SCAN algorithm moves towards 100 when it starts execution) is _____tracks

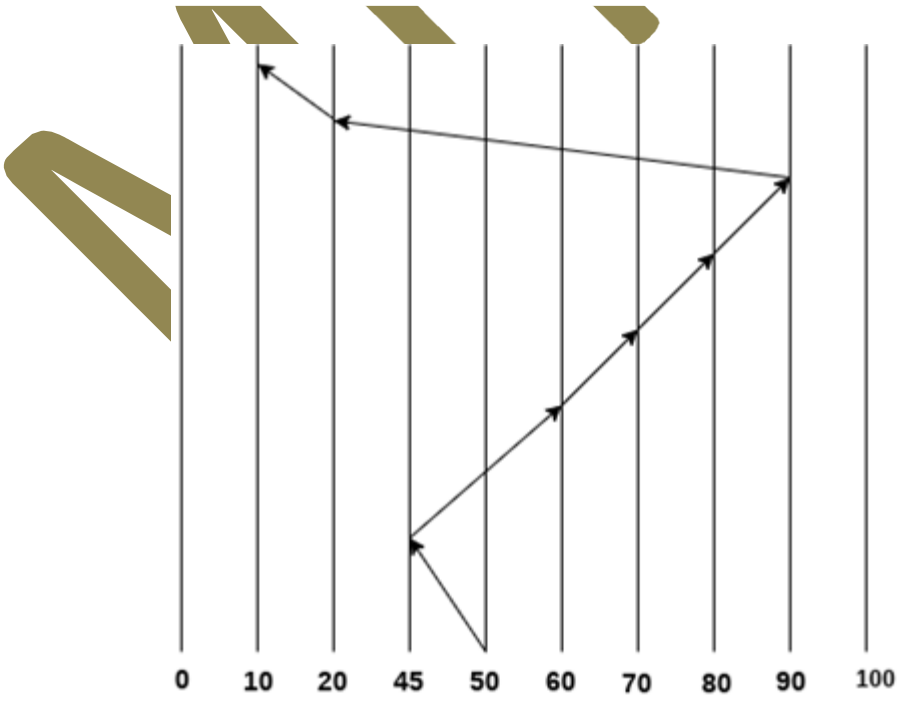
(A) 5
(B) 9
(C) 10
(D) 11

UsingSSTFAlgorithm

Numberoftrackare100.

InitialPositionofR/Wheadis 50.

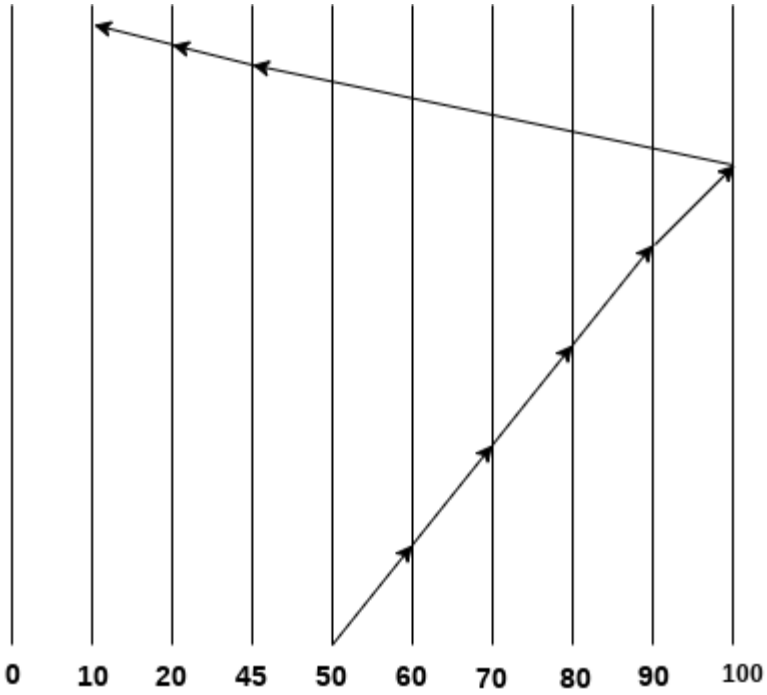
Therequestsare:45,20,90,10,50,60,80and 70



`

Numberofcrossedcylinders=5+ 15+10+10+10+70+10=130

UsingSCANAlgorithm



Numberofcylinders crosses=0+10+10+ 10+10+ 10+55+20+10=135

Thereforetheansweris(A).TheSCANalgorithmtravels for5additionaltracks.

-

`